

03_Assignment_Yuliia_Chernysheva

September 11, 2026

Yuliia Chernysheva
September 11, 2026
CSC 2267

1 CIS 2267 - Assignment 3 - Pandas Visualizations

There are numerous Python libraries for drawing charts. This lab uses Pandas, which was covered in the previous lab. Later, you'll use Seaborn which has more features and capabilities.

1.1 Read the data file into a DataFrame

*# <https://usafacts.org/data/topics/people-society/population-and-demographics/population-data/>
USAFacts grants you a license to use Original Content under the Creative Commons Attribution*

```
import pandas as pd
pop = pd.read_excel("population_usafacts.xlsx")
pop
```

```
[1]: import pandas as pd
pop = pd.read_excel("population_usafacts.xlsx")
pop
```

```
[1]:
```

	Year	Total	Alabama	Alaska	Arizona	Arkansas	California	\
0	1900	76094000	1830000	NaN	124000	1314000	1490000	
1	1901	77584000	1907000	NaN	131000	1341000	1550000	
2	1902	79163000	1935000	NaN	138000	1360000	1623000	
3	1903	80632000	1957000	NaN	144000	1384000	1702000	
4	1904	82166000	1978000	NaN	151000	1419000	1792000	
..	...	...	...	...	...	...	...	
116	2016	323071755	4866824	742575.0	6944767	2991815	39149186	
117	2017	325122128	4877989	740983.0	7048088	3003855	39337785	
118	2018	326838199	4891628	736624.0	7164228	3012161	39437463	
119	2019	328329953	4907965	733603.0	7291843	3020985	39437610	
120	2020	331501080	4921532	731158.0	7421401	3030522	39368078	

	Colorado	Connecticut	Delaware	...	South Dakota	Tennessee	Texas	\
0	543000	910000	185000	...	403000	2023000	3055000	
1	581000	931000	187000	...	418000	2041000	3132000	

2	621000	952000	188000	...	431000	2060000	3210000
3	652000	972000	190000	...	437000	2082000	3291000
4	659000	987000	192000	...	452000	2086000	3374000
..	...	...	...	...	...	...	...
116	5543844	3579830	949989	...	863693	6651277	27914064
117	5617421	3575324	957942	...	873732	6714748	28291024
118	5697155	3574561	966985	...	879386	6778180	28624564
119	5758486	3566022	976668	...	887127	6830325	28986794
120	5807719	3557006	986809	...	892717	6886834	29360759

	Utah	Vermont	Virginia	Washington	West Virginia	Wisconsin	Wyoming
0	277000	344000	1858000	523000	959000	2072000	93000
1	284000	347000	1887000	583000	972000	2109000	100000
2	292000	349000	1894000	651000	1000000	2141000	105000
3	299000	350000	1890000	719000	1037000	2171000	108000
4	308000	353000	1889000	782000	1064000	2202000	111000
..	...	...	...	...	...	...	...
116	3044241	624366	8417651	7299961	1832435	5775170	585243
117	3103540	625132	8471011	7427951	1818683	5793147	579994
118	3155153	624802	8510920	7526793	1805953	5809319	579054
119	3203383	624046	8556642	7614024	1795263	5824581	580116
120	3249879	623347	8590563	7693612	1784787	5832655	582328

[121 rows x 53 columns]

Set an index on the year column.

```
[2]: pop = pop.set_index('Year')
pop
```

```
[2]:
```

	Total	Alabama	Alaska	Arizona	Arkansas	California	Colorado	\
Year								
1900	76094000	1830000	NaN	124000	1314000	1490000	543000	
1901	77584000	1907000	NaN	131000	1341000	1550000	581000	
1902	79163000	1935000	NaN	138000	1360000	1623000	621000	
1903	80632000	1957000	NaN	144000	1384000	1702000	652000	
1904	82166000	1978000	NaN	151000	1419000	1792000	659000	
...	...	...	...	...	...	...	...	
2016	323071755	4866824	742575.0	6944767	2991815	39149186	5543844	
2017	325122128	4877989	740983.0	7048088	3003855	39337785	5617421	
2018	326838199	4891628	736624.0	7164228	3012161	39437463	5697155	
2019	328329953	4907965	733603.0	7291843	3020985	39437610	5758486	
2020	331501080	4921532	731158.0	7421401	3030522	39368078	5807719	

	Connecticut	Delaware	District of Columbia	...	South Dakota	\
Year				...		
1900	910000	185000		278000	...	403000

1901	931000	187000		285000	...	418000
1902	952000	188000		290000	...	431000
1903	972000	190000		295000	...	437000
1904	987000	192000		302000	...	452000
...	...	...		...	...	...
2016	3579830	949989		687576	...	863693
2017	3575324	957942		697079	...	873732
2018	3574561	966985		704147	...	879386
2019	3566022	976668		708253	...	887127
2020	3557006	986809		712816	...	892717

	Tennessee	Texas	Utah	Vermont	Virginia	Washington \
Year						
1900	2023000	3055000	277000	344000	1858000	523000
1901	2041000	3132000	284000	347000	1887000	583000
1902	2060000	3210000	292000	349000	1894000	651000
1903	2082000	3291000	299000	350000	1890000	719000
1904	2086000	3374000	308000	353000	1889000	782000
...	...	...	...	...	...	...
2016	6651277	27914064	3044241	624366	8417651	7299961
2017	6714748	28291024	3103540	625132	8471011	7427951
2018	6778180	28624564	3155153	624802	8510920	7526793
2019	6830325	28986794	3203383	624046	8556642	7614024
2020	6886834	29360759	3249879	623347	8590563	7693612

	West Virginia	Wisconsin	Wyoming
Year			
1900	959000	2072000	93000
1901	972000	2109000	100000
1902	1000000	2141000	105000
1903	1037000	2171000	108000
1904	1064000	2202000	111000
...	...	...	...
2016	1832435	5775170	585243
2017	1818683	5793147	579994
2018	1805953	5809319	579054
2019	1795263	5824581	580116
2020	1784787	5832655	582328

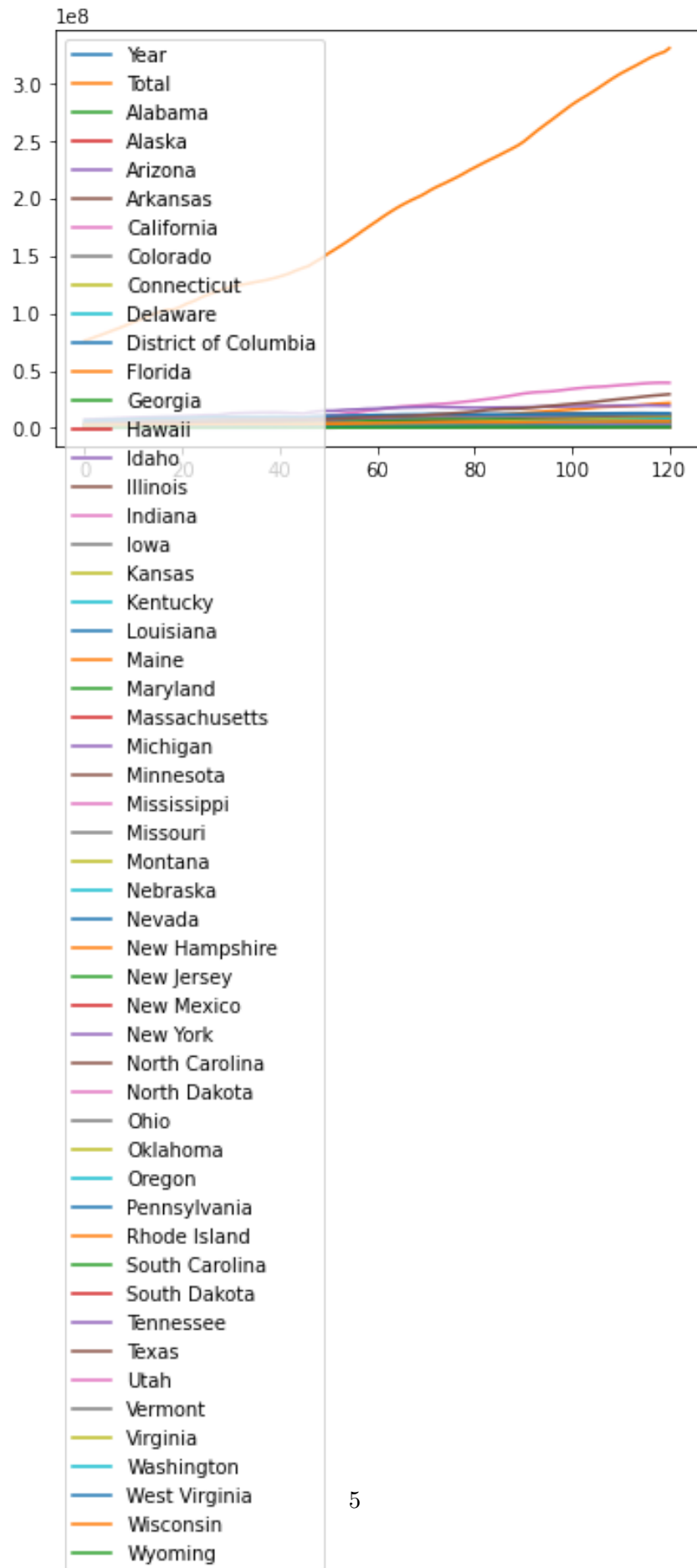
[121 rows x 52 columns]

1.2 Draw some plots

Start with the default options.

```
[3]: pop.reset_index().plot()
```

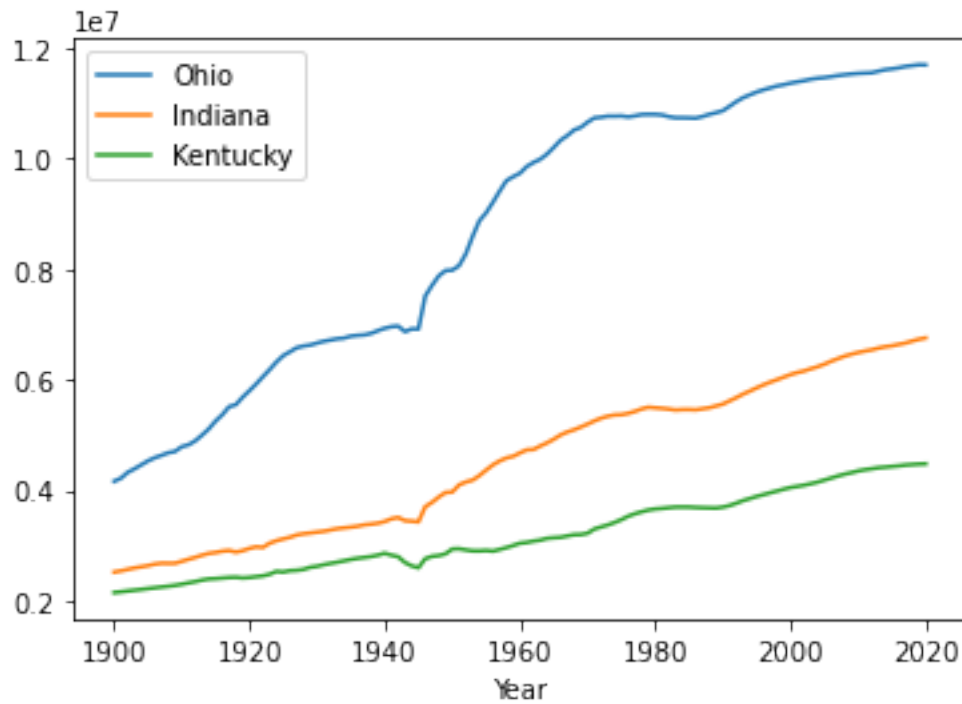
```
[3]: <AxesSubplot:>
```



Well, that's a mess, isn't it? Let's filter the data to get something more readable.

```
[4]: pop[['Ohio', 'Indiana', 'Kentucky']].plot.line()
```

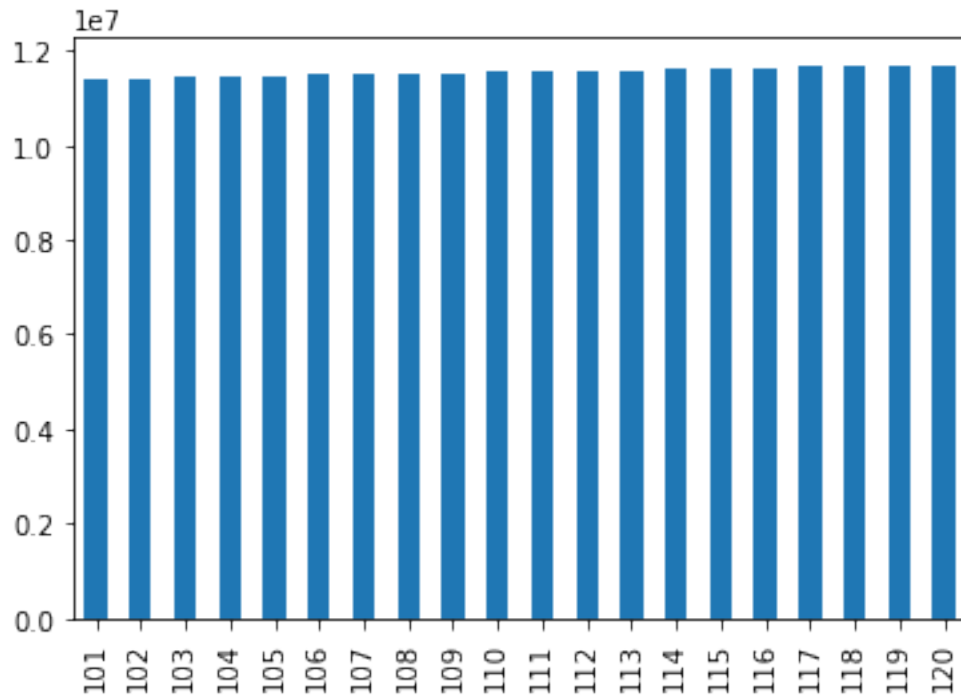
```
[4]: <AxesSubplot:xlabel='Year'>
```



Let's work with more recent times. Use a query to select years after 2000 and make a bar plot.

```
[5]: pop.reset_index().query('Year > 2000').Ohio.plot.bar()
```

```
[5]: <AxesSubplot:>
```

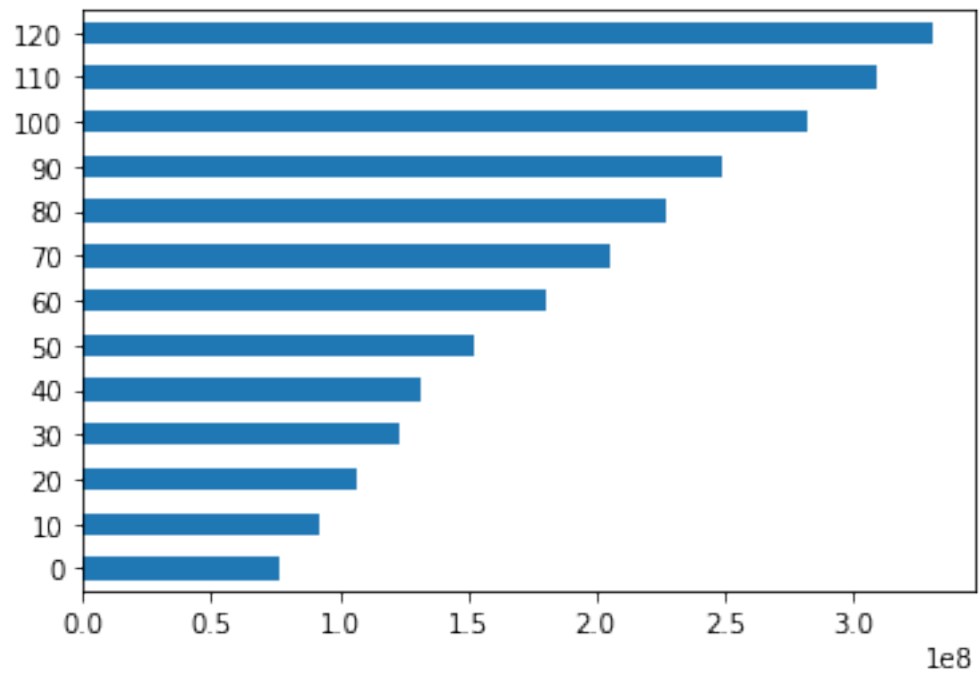


That's not very interesting– the values are mostly the same for that time period.

Make a range of the years 1900, 1910, 1920, etc. and make a horizontal bar plot.

```
[6]: pop.reset_index().query('Year % 10 == 0').Total.plot.barh()
```

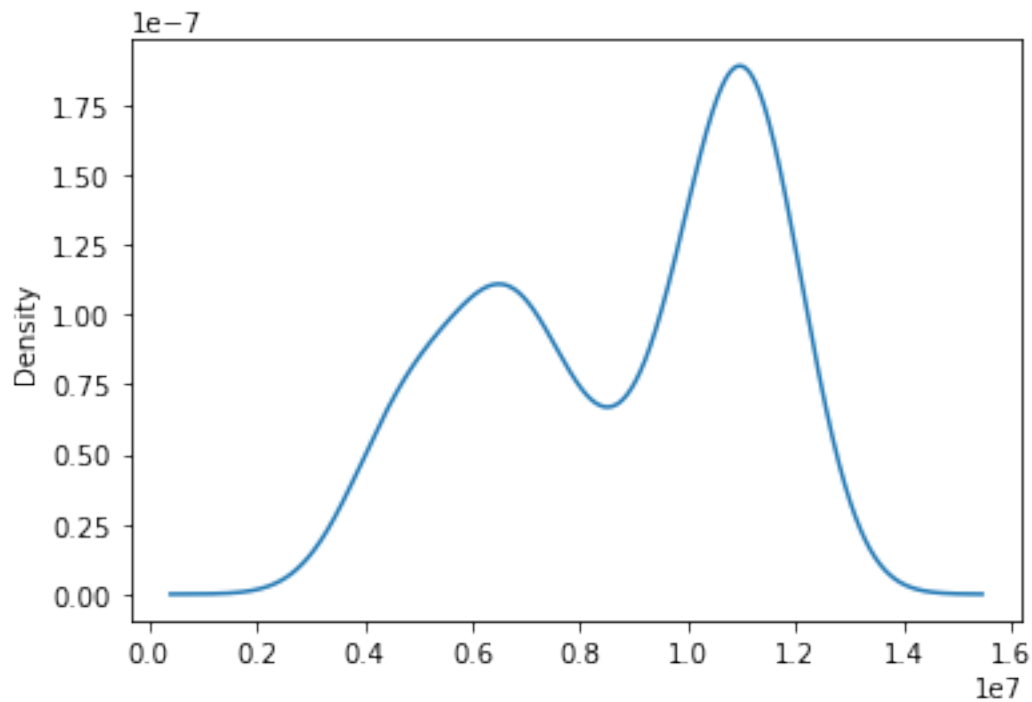
```
[6]: <AxesSubplot:>
```



Make a density plot of Ohio's population. This shows where most of the values are in the dataset

```
[7]: pop.Ohio.plot.density()
```

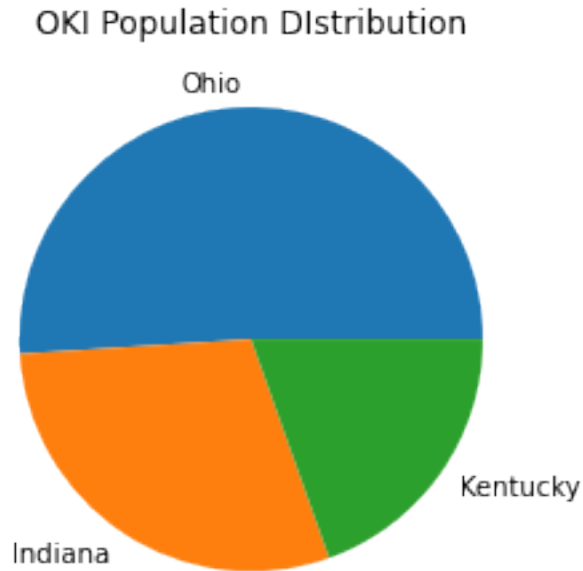
```
[7]: <AxesSubplot:ylabel='Density'>
```



Select the populations of Ohio, Indiana, and Kentucky for the year 2020, and draw a pie chart of the population distribution.

```
[8]: pop.loc[2020, ['Ohio', 'Indiana', 'Kentucky']].plot.pie(
      title = "OKI Population Distribution",
      ylabel = ''
    )
```

```
[8]: <AxesSubplot:title={'center':'OKI Population Distribution'}>
```



Make side-by-side subplots of the Ohio, Indiana, and Kentucky populations. Use the year as the x values, share the y axis, limit the years to 1990-2020. Add titles, and rotate the text of the x -axis.

```
[9]: pop[['Ohio', 'Indiana', 'Kentucky']].plot.line(
    title=['Ohio', 'Indiana', 'Kentucky'],
    xlabel='Year',
    ylabel='Population',
    sharey=True,
    rot = 45,
    xlim=(1990,2020),
    subplots=True,
    layout=(1,3)
)

[9]: array([[<AxesSubplot:title={'center':'Ohio'}, xlabel='Year',
ylabel='Population'>,
          <AxesSubplot:title={'center':'Indiana'}, xlabel='Year',
ylabel='Population'>,
          <AxesSubplot:title={'center':'Kentucky'}, xlabel='Year',
ylabel='Population'>]],
        dtype=object)
```

